Python Text File Handling Practice

10 Practice Problems Strictly Focused on Text Files (.txt) and Basic Modes: Read ('r'), Write ('w'), and Append ('a').

TEXT FILES (.TXT) READ MODE ('R') WRITE MODE ('W') APPEND MODE ('A')

Task 1: Line and Word Counter

Read Mode ('r')

PROBLEM STATEMENT

Write a Python program that reads an existing text file named `document.txt` and calculates total counts for both lines and words contained within the file.

FILE OPERATIONS REQUIRED

Open `document.txt` in Read mode (`'r'`), iterate through the lines, and compute counts.

INPUT TEXT FILE SETUP (DOCUMENT.TXT)

```
Hello world Welcome to Python file
handling Enjoy coding
```

EXPECTED CONSOLE OUTPUT

```
Total Lines: 3 Total Words: 8
```

Task 2: User Note Writer

Write Mode ('w')

PROBLEM STATEMENT

Create a Python program that takes N lines of text input from the console and writes them into a new file named `notes.txt`. Overwrite the file if it already exists.

FILE OPERATIONS REQUIRED

Open `notes.txt` in Write mode (`'w'`) and write the user inputs with newlines.

SAMPLE INPUT (CONSOLE)

```
3 Buy groceries Finish homework Call mom
```

EXPECTED FILE OUTPUT (NOTES.TXT)

```
Buy groceries Finish homework Call mom
```

Task 3: Activity Log Appender

Append Mode ('a')

PROBLEM STATEMENT

Develop a program that appends a single string entry given from user input to an existing log file named `activity.txt` without removing any existing logs.

FILE OPERATIONS REQUIRED

Open `activity.txt` in Append mode (`'a'`) and write the entry followed by a newline character.

EXISTING FILE + CONSOLE INPUT

```
Existing activity.txt: User logged in
Console Input: User updated profile
```

EXPECTED FILE OUTPUT (ACTIVITY.TXT)

```
User logged in User updated profile
```

Task 4: Text File Duplicator

Read ('r') & Write ('w')

PROBLEM STATEMENT

Write a Python script that reads content from a source file named `source.txt` and writes an exact copy into a new file named `destination.txt`.

FILE OPERATIONS REQUIRED

Open `source.txt` in Read mode (`'r'`) and `destination.txt` in Write mode (`'w'`).

INPUT TEXT FILE SETUP (SOURCE.TXT)

```
Line 1: Python programming Line 2: File modes demonstration
```

EXPECTED FILE OUTPUT (DESTINATION.TXT)

```
Line 1: Python programming Line 2: File modes demonstration
```

Task 5: Line Indexer File Writer

Read ('r') & Write ('w')

PROBLEM STATEMENT

Read an input text file `input.txt` line by line and write each line prefixed with its 1-based line number (e.g., `"1: "`) into a new text file `indexed.txt`.

FILE OPERATIONS REQUIRED

Read from `input.txt` using `'r'` and write formatted lines to `indexed.txt` using `'w'`.

INPUT TEXT FILE SETUP (INPUT.TXT)

```
Apple Banana Cherry
```

EXPECTED FILE OUTPUT (INDEXED.TXT)

```
1: Apple 2: Banana 3: Cherry
```

Task 6: Keyword Line Filter

Read ('r') & Write ('w')

PROBLEM STATEMENT

Read `raw_data.txt` and write only the lines that contain a user-specified search keyword into a new text file named `filtered.txt`.

FILE OPERATIONS REQUIRED

Open `raw_data.txt` in Read mode (`'r'`) and `filtered.txt` in Write mode (`'w'`).

INPUT FILE (RAW_DATA.TXT) + SEARCH TERM

```
raw_data.txt: [INFO] System started [ERROR] Database failed [INFO] User created [ERROR] Network timeout Console Input: ERROR
```

EXPECTED FILE OUTPUT (FILTERED.TXT)

```
[ERROR] Database failed [ERROR] Network timeout
```

Task 7: Case Inverter Writer

Read ('r') & Write ('w')

PROBLEM STATEMENT

Write a Python script that reads `story.txt`, converts all uppercase letters to lowercase and all lowercase letters to uppercase, and writes the output to `converted.txt`.

FILE OPERATIONS REQUIRED

Read `story.txt` using `'r'`, transform character cases, and write to `converted.txt` using `'w'`.

INPUT TEXT FILE SETUP (STORY.TXT)

```
Python File Handling IS Easy!
```

EXPECTED FILE OUTPUT (CONVERTED.TXT)

```
pYTHON fILE hANDLING is eASY!
```

Task 8: File Merger

Read ('r') & Append ('a')

PROBLEM STATEMENT

Write a Python program that reads contents from two separate text files, `part1.txt` and `part2.txt`, and appends them sequentially into a single target file `combined.txt`.

FILE OPERATIONS REQUIRED

Read `part1.txt` and `part2.txt` using `'r'`, and append their contents to `combined.txt` using `'a'`.

INPUT TEXT FILES SETUP

```
part1.txt: Header content part2.txt:
Footer content
```

EXPECTED FILE OUTPUT (COMBINED.TXT)

```
Header content Footer content
```

Task 9: Empty Line Remover

Read ('r') & Write ('w')

PROBLEM STATEMENT

Read a text file named `messy.txt` containing blank lines, remove all blank lines (or lines containing only whitespace), and save the cleaned output into `clean.txt`.

FILE OPERATIONS REQUIRED

Read from `messy.txt` with `'r'`, filter empty lines, and write non-empty lines to `clean.txt` using `'w'`.

INPUT TEXT FILE SETUP (MESSY.TXT)

```
First line Second line Third line
```

EXPECTED FILE OUTPUT (CLEAN.TXT)

```
First line Second line Third line
```

Task 10: Multi-Student Score Logger

Write ('w') & Append ('a')

PROBLEM STATEMENT

Design a student score tracker. First, write a header line `"Name, Score"` into a new file `scores.txt` using write mode. Then, run a loop that takes N student records from console input and appends each record line by line into `scores.txt` using append mode.

FILE OPERATIONS REQUIRED

Open `scores.txt` in Write mode (`'w'`) for header creation, then in Append mode (`'a'`) inside a loop for records.

SAMPLE INPUT (CONSOLE)

```
2 Alice, 92 Bob, 85
```

EXPECTED FILE OUTPUT (SCORES.TXT)

```
Name, Score Alice, 92 Bob, 85
```

Summary of Basic Text File Modes in Python

- **Read Mode (`'r'`):** Opens a text file for reading. Raises `FileNotFoundError` if the specified file path does not exist. Pointer starts at the beginning.
- **Write Mode (`'w'`):** Opens a text file for writing. Overwrites/truncates the file if it already exists, or creates a new file if it does not.
- **Append Mode (`'a'`):** Opens a text file for writing without erasing existing content. Appends new data at the end of the file or creates a new file if it does not exist.
- **Best Practice:** Always use context managers (`with open(...) as file:`) to ensure text file streams are closed automatically after processing.